

Challenge 2: Reconstruction and Optimization of a 256-bit Permutation

Crypto Contest Submission

17 July 2026

1 Summary

The recovered one-round permutation is

rotate $\rightarrow$ XOR $\rightarrow$ reverse all 32 bytes $\rightarrow$ add,

with word rotations

`rot = {43, 7, 29, 14}.`

The submitted C implementation passes all 1,000 supplied one-round vectors and the supplied 20-round vector. It replaces the generic byte-map loop with four 64-bit byte swaps and specializes the four rotations. A separate differential test compared the generic reference and optimized implementations on 100,000 deterministically generated states for both one and twenty rounds.

2 Recovering the structure

The four component operations are invertible. Word addition is inverted by subtraction modulo 2^{64} ; XOR is self-inverse; the map `shuffle_map[i] = 31-i` is also self-inverse; and only 63 non-zero rotation values are permitted.

For each supplied input/output pair, I computed

```
t0 = output - constants1 // modulo  $2^{64}$  per word
t1 = reverse_all_32_bytes(t0)
t2 = t1 XOR constants2
```

and tested every r in $1, \dots, 63$ for

```
ROTL64(input[i], r) == t2[i].
```

The recovery script intersects the candidate sets from all 1,000 vectors rather than relying on one possibly degenerate input word. Each word has exactly one candidate: 43, 7, 29, and 14. Forward evaluation then confirms the stated operation order on every supplied vector.

3 Reference implementation

The direct restoration calls the four supplied operations in sequence:

```
rotate_words_left_64wise(state, rot);
xor_constants_256wise(state, constants2);
shuffle_bytes_256(state, shuffle_map);
add_constants_64wise(state, constants1);
```

The generic shuffle copies the state into a 32-byte temporary array, performs 32 indexed byte loads and stores, and copies the result back. This is correct for an arbitrary map but unnecessarily general for the fixed contest map.

4 Optimized implementation

4.1 Replacing the byte map

Reversing all bytes of four 64-bit words is equivalent to reversing the word order and byte-swapping each word:

```
reverse32(x0,x1,x2,x3) =  
    (BSWAP64(x3), BSWAP64(x2), BSWAP64(x1), BSWAP64(x0)).
```

The submission uses `__builtin_bswap64` with GCC or Clang. It also contains a portable shift-and-mask fallback. This removes the temporary arrays, the map lookups, and most of the shuffle memory traffic.

4.2 Constant rotations and scalar registers

Combining the recovered rotations with the word reversal gives the complete round directly:

```
y0 = BSWAP64(ROTL64(x3,14) ^ C2[3]) + C1[0]  
y1 = BSWAP64(ROTL64(x2,29) ^ C2[2]) + C1[1]  
y2 = BSWAP64(ROTL64(x1, 7) ^ C2[1]) + C1[2]  
y3 = BSWAP64(ROTL64(x0,43) ^ C2[0]) + C1[3]
```

The fixed shift counts permit immediate-count rotate/shift instructions. The four state words remain scalar locals across the round calculation. The code is portable C and does not require AVX2. A single state is only four words, each lane uses a different rotation, and rounds are serially dependent, so the scalar transformation avoids SIMD setup and lane-handling overhead.

The problem statement allows external helper functions. The original test and timing code is unchanged. The one-round TODO uses the four restored generic operations, the rotation TODO sets the recovered array, and the performance- critical 20-round TODO invokes the specialized helper in its loop.

5 Correctness testing

The following tests passed:

- all 1,000 supplied one-round input/output pairs;
- the supplied 20-round input/output pair;
- 100,000 deterministic random states, comparing reference and optimized output after one round and after twenty rounds; and
- the original contest test harness compiled with the final submission.

The supplied 20-round test is

```
input  
0011223344556677 8899aabbccddeeff  
0123456789abcdef fedcba9876543210  
output  
7527ddf4dcac1a6c c16754941f41bbba  
5f35b110c1dc4504 2d6663801ac4aae8
```

Observed validation output was:

```
one_round_vectors=1000  
twenty_round_vector=PASS  
random_differential_cases=100000  
selftest=PASS
```

6 Benchmark methodology and results

The reproducible benchmark compiles the generic reference and optimized scalar implementation in the same translation unit with

```
gcc -O3 -std=c11 -Wall -Wextra -Wpedantic -DNDEBUG
```

No `-march=native` option was used. The timer is `clock_gettime(CLOCK_MONOTONIC)`. After a warm-up, each sample performs 1,000,000 calls of the 20-round function. Reference-first and optimized-first order alternates between samples. Nine samples are collected and the median wall-clock time of each implementation is reported. The final state is consumed through a volatile sink.

The measurement host was a Linux x86-64 VM with an AMD EPYC 7B12 and GCC 12.2.0. Results measured on 17 July 2026 were:

Implementation	ns per 20 rounds	Relative performance
Generic reference	834.819	1.000x
Optimized scalar	44.101	18.930x

The unmodified contest timing harness reported 0.049428 microseconds per 20-round call in one additional run. The table uses the more robust alternating nine-sample median. These numbers compare implementations on this VM; the absolute time and speedup may change on the contest Intel Core Ultra 7 255H with GCC 13.3.0.

7 Reproduction

From the repository root, reconstruction and Python verification use:

```
python3 solutions/solve_02_permutation.py
```

The C differential test and benchmark use:

```
python3 solutions/benchmark_02_permutation.py \  
--iterations 1000000 --repeats 9 --random-cases 100000
```

For submission testing, place `contest.c` beside the two supplied vector files and run:

```
gcc -O3 -Wall -Wextra -o contest contest.c  
./contest
```